NETSAGE AI

Project Summary Documentation

1. Project Overview

NetSage AI is an evidence-based Cisco-style network troubleshooting and diagnosis platform. The system takes a predefined network case containing a symptom, topology/device information, and supplied command output/evidence. It processes the case through a deterministic rule checker and an AI diagnosis layer, followed by human validation and responsible-AI audit logging.

The project is designed to demonstrate how traditional rule-based network troubleshooting can be combined with AI-assisted reasoning while retaining human oversight. The dashboard provides a single interface for viewing cases, rule-checker findings, AI diagnoses, human decisions, evaluation results, and audit records.

2. Important Note About the Network / Packet Tracer Data

The network topology and command outputs used by NetSage AI are simulated laboratory evidence. The project does not rely on a live physical Cisco network, real production devices, or a real Packet Tracer (.pkt) topology as the source of truth.

The cases.csv dataset contains predefined network symptoms, topology/device descriptions, and show-command-style evidence. Therefore, the displayed command outputs and troubleshooting scenarios should be described as simulated/test-lab evidence rather than real network telemetry. This is an intentional limitation of the current prototype.

3. Problem Statement

Network troubleshooting often requires identifying the source of connectivity failures from symptoms, topology information, and device command output. Beginners may focus on the first possible explanation and can miss configuration relationships across VLANs, routing, DHCP, DNS, ACLs, NAT, interfaces, and other networking layers.

NetSage AI addresses this by combining deterministic checks with AI-assisted diagnosis and human review. The goal is not to blindly accept an AI answer, but to create an evidence-based troubleshooting workflow in which each diagnosis can be examined, corrected, rejected, and logged.

4. Objectives

- Build a structured dataset of network troubleshooting cases.
- Implement deterministic rules for common network faults.
- Use an AI model to reason over the supplied evidence and propose a root cause.
- Prevent unsupported assumptions by instructing the AI not to invent network information.
- Provide a human-in-the-loop review mechanism.
- Maintain a responsible-AI audit trail of human decisions.

- Evaluate case coverage across the complete processing pipeline.
- Provide a professional Streamlit dashboard and a live prototype.

5. System Architecture / Workflow

1. **Case Dataset** – stores the symptom, topology/device information, command output/evidence, expected fault, OSI layer, concept, and severity.
2. **Deterministic Rule Checker** – examines evidence for recognizable faults such as gateway mismatch, missing route, wrong next hop, VLAN problems, trunk issues, DHCP/DNS failures, ACL problems, NAT problems, interface failures, and duplicate IPs.
3. **AI Diagnosis** – sends the case evidence and checker findings to the Groq-hosted model using the diagnosis prompt. The model produces a structured troubleshooting diagnosis.
4. **Human Review** – a reviewer compares the AI diagnosis with the supplied evidence and selects Approved, Corrected, or Rejected.
5. **Responsible AI Log** – records the AI diagnosis, human decision, corrected diagnosis when applicable, reason/comment, and action taken.
6. **Evaluation** – checks case counts, stage coverage, human-review outcomes, missing cases, and responsible-AI logging.
7. **Dashboard** – presents the complete evidence → rules → AI → human → audit workflow through Streamlit.

6. Team Contribution

The four members' work is interconnected rather than isolated. The data and integration layer provides the cases consumed by the checker; the checker produces findings used by the AI; the AI produces diagnoses that are reviewed by a human; the review generates responsible-AI audit records; and the dashboard and evaluation modules bring the complete workflow together.

Sno.	Team Member Name	Enrollment Number	College Name	Role	Description
1.	Parth Sharma	0157AL241108	LNCT Group of College	AI Diagnosis and Prompt Engineering	Building the AI reasoning layer that converts network evidence and rule findings into a structured troubleshooting diagnosis.
2.	Ritesh Singh Mandloi	0157AL241143	LNCT Group of College	Rule-Based Network Diagnostics and Backend Logic	Building the deterministic network troubleshooting engine that provides explainable rule-based evidence before AI reasoning.
3.	Payal Baghel	0157AL241111	LNCT Group of College	Frontend, Human Review and Responsible AI	Building the user-facing dashboard and human oversight mechanism that allows users to inspect,

					validate, correct, and audit AI diagnoses.
4.	Radhika Patil	0157AL241133	LNCT Group of College	Project Integration, Data Pipeline, Evaluation & Deployment	Ensuring that the individual project modules function as one complete end-to-end NetSage AI system.

7. Technology Stack

- Python – core implementation language
- Pandas – CSV/data processing and analysis
- Streamlit – dashboard, human-review interface, and live prototype
- Plotly – dashboard charts and visualizations
- Groq API – AI diagnosis service
- Git/GitHub – source-code version control and project collaboration
- CSV – lightweight project data and audit storage
- Markdown – AI prompt documentation

8. Major Project Components

Component	Purpose
cases.csv	Main test dataset containing the simulated network troubleshooting cases.
checker.py	Rule-based diagnostic engine that applies deterministic checks to case evidence.
checker_results.csv	Stores the findings generated by the deterministic checker.
diagnose_prompt.md	Defines the AI's role, troubleshooting methodology, evidence rules, response structure, and restrictions against invented evidence.
diagnosis.py	Loads cases and checker findings, builds the AI prompt, calls the Groq model, and stores AI diagnoses in ai_results.csv.
ai_results.csv	Stores the generated AI diagnosis for each case.
review_app.py	Streamlit human-review interface for comparing AI output with evidence and recording reviewer decisions.
human_reviews.csv	Stores human approval, correction, or rejection records.
responsible_ai_log.csv	Maintains the audit trail for human oversight and actions taken after reviewing AI output.
evaluate.py	Performs pipeline coverage and human-review evaluation checks.
dashboard.py	Professional Streamlit dashboard containing Dashboard, Case Analysis, Evaluation, and Responsible AI sections.

requirements.txt	Lists the Python dependencies needed for the deployed Streamlit application.

9. AI Diagnosis Methodology

The AI is instructed to act as a Cisco-style network troubleshooting assistant. For each case it receives the case ID, symptom, topology/device information, supplied command output, and deterministic rule-checker findings.

- Identify the source and destination.
- Follow the likely network path.
- Examine the supplied command output.
- Identify incorrect or missing information supported by the evidence.
- Determine the most likely root cause.
- Identify the relevant OSI layer and networking concept.
- Recommend a next command that would confirm the diagnosis.
- Suggest a fix and explain how to verify it.
- Return 'Insufficient evidence' when the supplied evidence is not enough.

The prompt explicitly prohibits inventing IP addresses, VLANs, routes, ACL entries, device configurations, or command output. The AI is also instructed not to blindly accept the deterministic rule checker.

10. Deterministic Rule Checker

The current checker contains rules covering:

- Gateway mismatch
- Missing route
- Wrong next hop
- Missing/wrong VLAN assignment and wireless VLAN
- Trunk VLAN allowance
- DHCP pool, DHCP relay, pool exhaustion, and wireless DHCP issues
- DNS failure
- ACL blocking, ICMP/HTTP restrictions, ACL direction, and guest isolation
- NAT/PAT configuration and NAT ACL issues
- Administratively down or up/down interfaces
- Duplicate IP addresses

The checker is deterministic: it looks for specific patterns in the supplied evidence and records matching rules. It is therefore a validation/diagnostic aid, not a replacement for the AI or human review.

11. Human-in-the-Loop and Responsible AI

A central design principle of NetSage AI is that AI output is not treated as automatically correct. The human reviewer can approve the AI diagnosis, correct it, or reject it.

- Approved – the reviewer accepts the AI diagnosis.

- Corrected – the reviewer believes the diagnosis is wrong or incomplete and supplies the corrected diagnosis.
- Rejected – the reviewer rejects the AI diagnosis.

Every saved review is also reflected in the responsible_ai_log.csv audit file. This provides traceability between the AI output and the final human decision.

12. Dashboard and Prototype

The Streamlit dashboard provides four main sections:

- Dashboard – displays total cases, rule coverage, AI coverage, human-review counts, pipeline charts, severity distribution, network problem categories, and the case dataset.
- Case Analysis – displays an individual case's symptom, topology, evidence, expected fault, deterministic findings, AI diagnosis, and human validation.
- Evaluation – displays pipeline coverage, human-validation outcomes, and evaluation summaries.
- Responsible AI – displays the human-in-the-loop architecture, oversight outcomes, responsible-AI principles, and audit log.

The Streamlit application has been deployed as the project's live prototype. The deployed URL can be used as the prototype/demo link, while the GitHub repository provides the source code.

13. Evaluation

The evaluation script checks whether the expected number of cases are present across the rule-checker, AI diagnosis, human-review, and responsible-AI stages. It also reports approval, correction, and rejection rates and identifies missing case IDs.

A human approval result should not be interpreted as independently verified AI accuracy. True model accuracy would require an independently established ground-truth evaluation set and a defined scoring method.

14. Limitations

- The network cases and command outputs are simulated test-lab evidence rather than live network telemetry.
- The current project does not directly connect to real Cisco devices for automatic command collection.
- The deterministic checker recognizes predefined evidence patterns and does not cover every possible network fault.
- AI output can still be incorrect; therefore, human review remains an important part of the workflow.
- The current evaluation primarily measures pipeline completeness and human-review outcomes rather than independently verified model accuracy.
- CSV files are suitable for a prototype but would normally be replaced by a database and stronger access controls in a production system.

15. Project Outcome

NetSage AI successfully demonstrates a complete prototype for **evidence-based network troubleshooting using deterministic rules, AI reasoning, and human oversight**.

The project provides:

- A structured network case dataset
- Deterministic diagnostic rules
- AI-assisted diagnosis
- Evidence-constrained prompting
- Human review
- Responsible-AI logging
- Pipeline evaluation
- Interactive dashboard
- GitHub source repository
- Live Streamlit deployment

The primary contribution of the project is not simply the use of AI for network troubleshooting. Instead, it demonstrates an approach where **AI assistance is combined with deterministic validation, explicit evidence constraints, human judgment, and auditability**.

16. Final Deliverables

- Source code repository on GitHub
  https://github.com/radhika2406/NetSage-AI.git
- Live Streamlit prototype/deployment URL
  https://netsage-ai-kbfp4x58yhnhzqzqveuaxu.streamlit.app/
- Demo Video link
  https://drive.google.com/file/d/1gtgaDFi68keyB52Yr9_uD_OQKH62de-N/view?usp=sharing